

Name : Anmol Sharma

Section : 2A

University Roll No. : 2415500076

Class Roll No. : 10

Subject : Probability and Statistics

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
```

1. Impact of Screen Time on Sleep Quality (Correlation) Generate 250 synthetic records with screen_time, sleep_duration, weekend flag. Compute Pearson, scatter plot, and interpret.

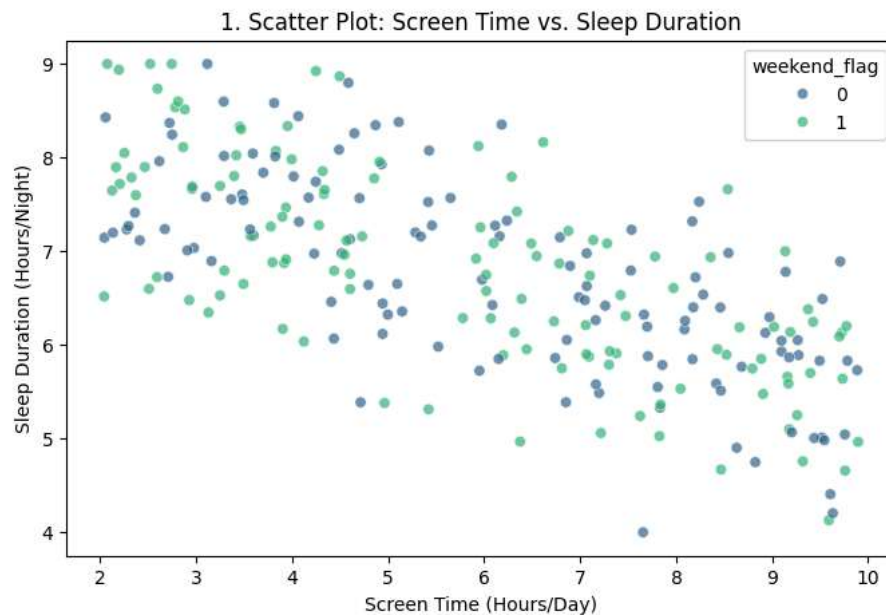
```
np.random.seed(42)

N1 = 250
screen_time = np.random.uniform(2.0, 10.0, N1)
# Simulate negative correlation
noise = np.random.normal(0, 0.8, N1)
sleep_duration = 8.5 - 0.3 * screen_time + noise
sleep_duration = np.clip(sleep_duration, 4.0, 9.0)

df1 = pd.DataFrame({
    'screen_time': screen_time,
    'sleep_duration': sleep_duration,
    'weekend_flag': np.random.randint(0, 2, N1)
})
print("--- Data Sample (Screen Time) ---")
print(df1.head())
```

```
--- Data Sample (Screen Time) ---
   screen_time  sleep_duration  weekend_flag
0    4.996321    6.323669           0
1    9.605714    4.406408           0
2    7.855952    5.786003           0
3    6.789268    7.148339           0
4    3.248149    7.696830           1
```

```
# Assuming df1 is loaded
plt.figure(figsize=(8, 5))
sns.scatterplot(x='screen_time', y='sleep_duration', hue='weekend_flag', data=df1, palette='viridis', alpha=0.7)
plt.title('1. Scatter Plot: Screen Time vs. Sleep Duration')
plt.xlabel('Screen Time (Hours/Day)')
plt.ylabel('Sleep Duration (Hours/Night)')
plt.show()
```



2. Predicting House Rent Using Building Features (Multiple Regression) Generate 600 records with area, rooms, distance from metro, zone, and rent. Fit regression and evaluate.

```
np.random.seed(42)

N2 = 600
area = np.random.uniform(800, 3000, N2)
rooms = np.random.randint(1, 6, N2)
dist_metro = np.random.uniform(0.5, 10.0, N2)
zone = np.random.choice(['A', 'B', 'C'], N2, p=[0.3, 0.4, 0.3])

# Rent calculation simulates effects: +Area, +Rooms, -Distance, +Zone A/B
rent_noise = np.random.normal(0, 500, N2)
rent = (1500 + 2.5 * area + 500 * rooms - 150 * dist_metro +
        (np.where(zone == 'A', 2500, np.where(zone == 'B', 1000, 0)))) +
        rent_noise
rent = np.clip(rent, 500, 15000)

df2 = pd.DataFrame({
    'area': area,
    'rooms': rooms,
    'distance_from_metro': dist_metro,
    'zone': zone,
    'rent': rent
})
print("--- Data Sample (House Rent) ---")
print(df2.head())
```

```
--- Data Sample (House Rent) ---
   area  rooms  distance_from_metro zone      rent
0  1623.988261    5         4.467017    B  8084.033503
1  2891.571474    2         8.851624    C  8382.666698
2  2410.386672    4         9.474954    A 10390.072436
3  2117.048665    2         4.940314    C  6705.364020
4  1143.241009    5         6.327408    C  5205.832561
```

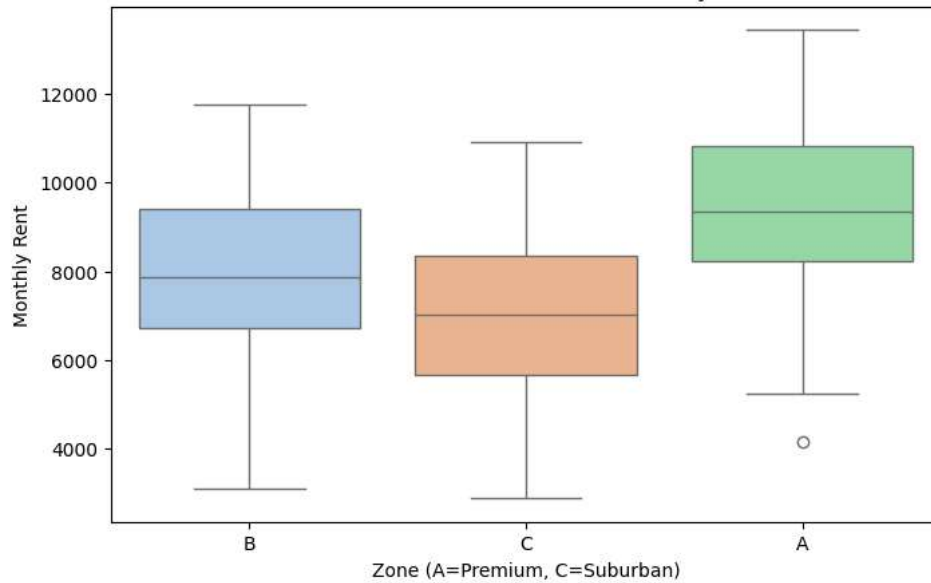
```
# Assuming df2 is loaded
plt.figure(figsize=(8, 5))
sns.boxplot(x='zone', y='rent', data=df2, palette='pastel')
plt.title('2. Box Plot: House Rent Distribution by Zone')
plt.xlabel('Zone (A=Premium, C=Suburban)')
plt.ylabel('Monthly Rent')
plt.show()
```

```
/tmp/ipython-input-3540430755.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.boxplot(x='zone', y='rent', data=df2, palette='pastel')
```

2. Box Plot: House Rent Distribution by Zone



3. Effect of Discounts on Online Sales (Regression) Generate 400 products with discount and units_sold (Poisson). Fit linear and quadratic models.

```
np.random.seed(42)
```

```
N3 = 400
discount = np.random.uniform(0.0, 0.50, N3)
# Simulates non-linear, accelerating sales response to discount
lambda_param = np.exp(2 + 4 * discount + 10 * discount**2)
units_sold = np.random.poisson(lambda_param, N3)
```

```
df3 = pd.DataFrame({
    'discount': discount,
    'units_sold': units_sold
})
print("--- Data Sample (Discounts & Sales) ---")
print(df3.head())
```

```
--- Data Sample (Discounts & Sales) ---
discount  units_sold
0  0.187270         22
1  0.475357        462
2  0.365997        118
3  0.299329         72
4  0.078009         18
```

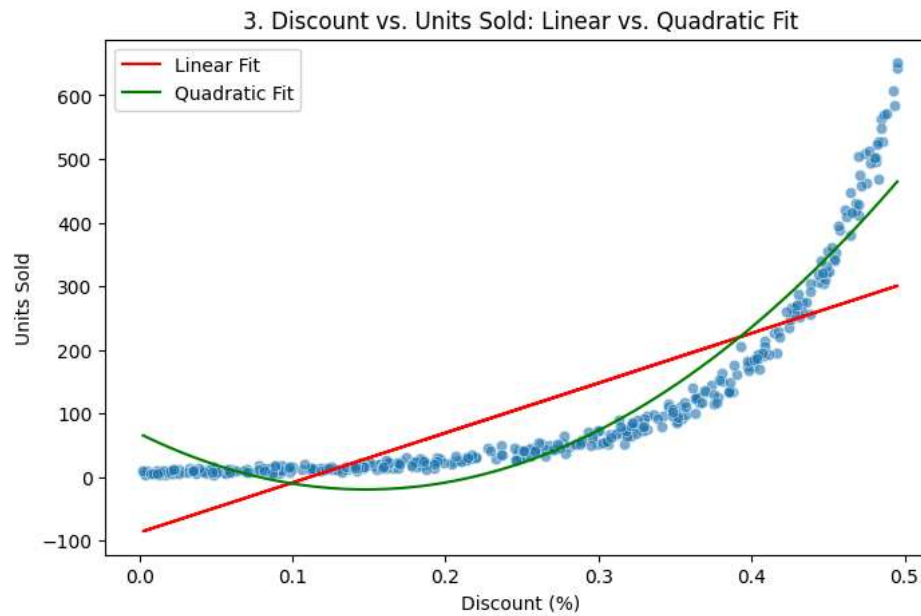
```
from statsmodels.formula.api import ols
```

```
# Assuming df3 is loaded
# Fit models to get prediction lines
model3_lin = ols('units_sold ~ discount', data=df3).fit()
model3_quad = ols('units_sold ~ discount + I(discount**2)', data=df3).fit() # I() for squared term
```

```
plt.figure(figsize=(8, 5))
sns.scatterplot(x='discount', y='units_sold', data=df3, alpha=0.6)
# Plot predictions for Linear Model
plt.plot(df3['discount'], model3_lin.predict(df3), color='red', label='Linear Fit')
# Plot predictions for Quadratic Model
plot_data = df3.sort_values('discount')
plt.plot(plot_data['discount'], model3_quad.predict(plot_data), color='green', label='Quadratic Fit')
```

```
plt.title('3. Discount vs. Units Sold: Linear vs. Quadratic Fit')
plt.xlabel('Discount (%)')
plt.ylabel('Units Sold')
```

```
plt.legend()
plt.show()
```



4. Hypothesis Test on Gender & Learning Mode (Chi-square) Generate 400 respondents with gender and online/offline preference. Perform chi-square test.

```
np.random.seed(42)

N4 = 400
gender = np.random.choice(['Male', 'Female'], N4)
# Simulates Females having a slightly higher preference for Online (60% vs 40%)
p_online = np.where(gender == 'Female', 0.6, 0.4)
preference = np.where(np.random.rand(N4) < p_online, 'Online', 'Offline')

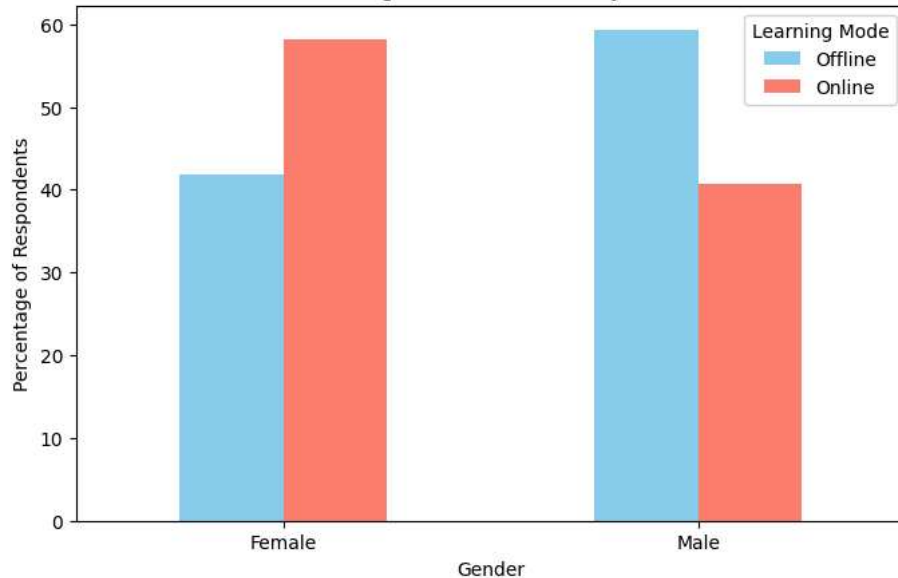
df4 = pd.DataFrame({
    'gender': gender,
    'preference': preference
})
print("--- Data Sample (Chi-square) ---")
print(df4.head())
```

```
--- Data Sample (Chi-square) ---
gender preference
0    Male   Offline
1  Female   Online
2    Male   Online
3    Male   Offline
4    Male   Offline
```

```
# Assuming df4 is loaded
# Use the proportion of preferences within each gender
preference_prop = df4.groupby('gender')['preference'].value_counts(normalize=True).mul(100).unstack()

preference_prop.plot(kind='bar', stacked=False, figsize=(8, 5), color=['skyblue', 'salmon'])
plt.title('4. Learning Mode Preference by Gender')
plt.ylabel('Percentage of Respondents')
plt.xlabel('Gender')
plt.xticks(rotation=0)
plt.legend(title='Learning Mode')
plt.show()
```

4. Learning Mode Preference by Gender



5. Temperature vs Electricity Consumption (Regression + Seasonality) Generate 365 daily temperature and electricity usage. Fit regression and identify patterns.

```
np.random.seed(42)

N5 = 365
# Generate temperatures varying over a year
temperature = 15 + 10 * np.sin(np.linspace(0, 2 * np.pi, N5)) + np.random.normal(0, 3, N5)
optimal_temp = 15
temp_diff = temperature - optimal_temp
# Simulates U-shaped (quadratic) usage
usage_base = 50
electricity_usage = usage_base + 0.8 * temp_diff**2 + np.random.normal(0, 10, N5)
electricity_usage = np.clip(electricity_usage, 20, 200)

df5 = pd.DataFrame({
    'temperature': temperature,
    'electricity_usage': electricity_usage
})
print("--- Data Sample (Temperature & Electricity) ---")
print(df5.head())
```

```
--- Data Sample (Temperature & Electricity) ---
   temperature  electricity_usage
0    16.490142         47.764215
1    14.757814         52.287848
2    17.288227         54.314710
3    20.086703         71.676400
4    14.987451         42.270028
```

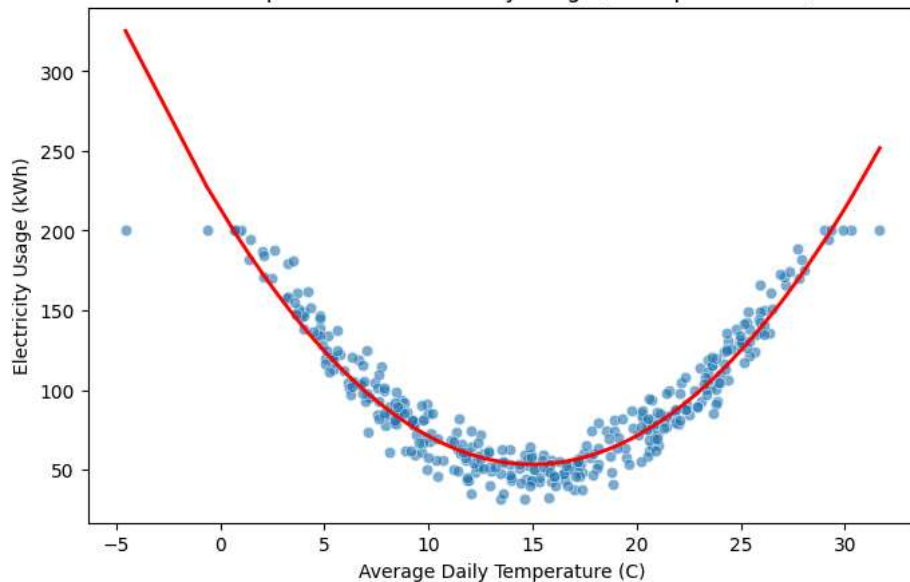
```
from statsmodels.formula.api import ols

# Assuming df5 is loaded
model5 = ols('electricity_usage ~ temperature + I(temperature**2)', data=df5).fit()

plt.figure(figsize=(8, 5))
sns.scatterplot(x='temperature', y='electricity_usage', data=df5, alpha=0.6)
# Plot the U-shaped quadratic fit
plot_data = df5.sort_values('temperature')
plt.plot(plot_data['temperature'], model5.predict(plot_data), color='red', linewidth=2)

plt.title('5. Temperature vs. Electricity Usage (U-shaped Pattern)')
plt.xlabel('Average Daily Temperature (C)')
plt.ylabel('Electricity Usage (kWh)')
plt.show()
```

5. Temperature vs. Electricity Usage (U-shaped Pattern)



7. Salary Prediction (Multicollinearity) Generate 300 records with experience, certifications, salary. Fit regression.

```
np.random.seed(42)

N7 = 300
experience = np.random.uniform(1, 20, N7)
# Simulate high correlation between experience and certifications
certs_noise = np.random.normal(0, 1, N7)
certifications = (1 + 0.4 * experience + certs_noise)
certifications = np.clip(np.round(certifications), 0, 10)

# Salary depends on both
salary_noise = np.random.normal(0, 8000, N7)
salary = 40000 + 4000 * experience + 1500 * certifications + salary_noise
salary = np.clip(salary, 40000, 200000)

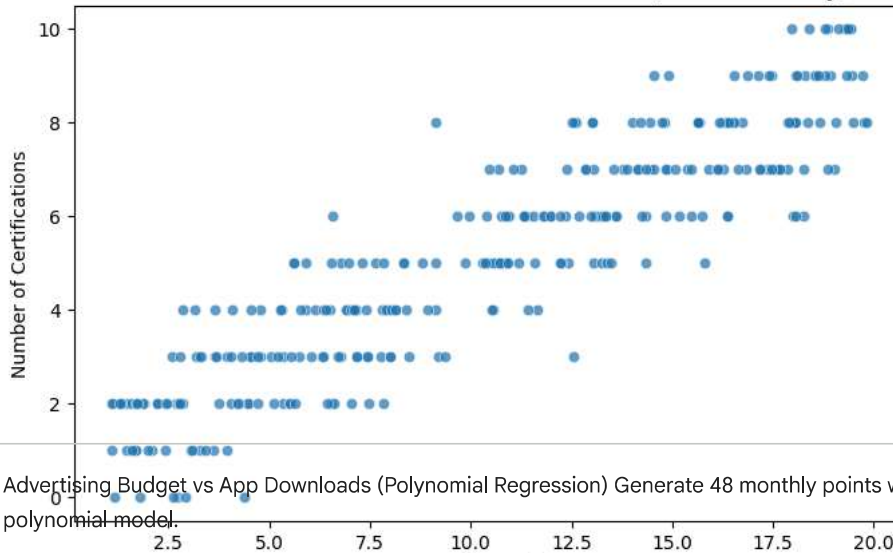
df7 = pd.DataFrame({
    'experience': experience,
    'certifications': certifications,
    'salary': salary
})
print("--- Data Sample (Salary Prediction) ---")
print(df7.head())
```

```
--- Data Sample (Salary Prediction) ---
   experience  certifications      salary
0    8.116262             4.0  77936.410643
1   19.063572             8.0 118566.157689
2   14.907885             9.0 107916.850715
3   12.374511             7.0 100377.234170
4    3.964354             1.0  50474.109751
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming df7 is loaded
plt.figure(figsize=(8, 5))
sns.scatterplot(x='experience', y='certifications', data=df7, alpha=0.7)
plt.title('7. Scatter Plot: Correlation between Predictors (Multicollinearity)')
plt.xlabel('Years of Experience')
plt.ylabel('Number of Certifications')
plt.show()
```

7. Scatter Plot: Correlation between Predictors (Multicollinearity)



9. Advertising Budget vs App Downloads (Polynomial Regression) Generate 48 monthly points with ad_budget and downloads. Fit polynomial model.

```
np.random.seed(42)

N9 = 48
ad_budget = np.linspace(10, 500, N9) # Budget in thousands
# Simulates diminishing returns: linear term is positive, quadratic term is negative
download_noise = np.random.normal(0, 500, N9)
downloads = (5000 + 100 * ad_budget - 0.1 * ad_budget**2 + download_noise)
downloads = np.clip(downloads, 1000, 15000)

df9 = pd.DataFrame({
    'ad_budget': ad_budget,
    'downloads': downloads
})
print("--- Data Sample (Ad Budget) ---")
print(df9.head())
```

```
--- Data Sample (Ad Budget) ---
   ad_budget  downloads
0    10.000000  6238.357077
1    20.425532  6931.700806
2    30.851064  8313.771838
3    41.276596  9718.798767
4    51.702128  9785.825078
```

```
from statsmodels.formula.api import ols

# Assuming df9 is loaded
model9 = ols('downloads ~ ad_budget + I(ad_budget**2)', data=df9).fit()

plt.figure(figsize=(8, 5))
sns.scatterplot(x='ad_budget', y='downloads', data=df9, alpha=0.7)
# Plot the concave-down polynomial fit
plot_data = df9.sort_values('ad_budget')
plt.plot(plot_data['ad_budget'], model9.predict(plot_data), color='red', linewidth=2)

plt.title('9. Ad Budget vs. App Downloads (Diminishing Returns)')
plt.xlabel('Ad Budget (in thousands)')
plt.ylabel('App Downloads')
plt.show()
```

